Crossover on a Real Neural Architecture Search Space: Structure Does Not Help, and at Scale It Hurts

Vasili Gavrilov*

2026

Abstract

Evolutionary neural architecture search recombines architectures with crossover, yet random search is famously hard to beat and it is rarely clear whether crossover contributes anything of its own. We test this on NAS-Bench-101, a tabulated space of 423,624 cells whose fitness is a lookup, so thousands of paired seeds are cheap. Two findings, both with paired significance tests. (i) Crossover *does* beat random search here, significantly at every budget. (ii) Making the crossover *respect the space's structure makes it worse.* The cell is multi-dimensional — wiring (adjacency) × labeling (operations), plus a path view — so we recombine along those axes: factor wiring from labeling, align nodes by role, transplant whole paths. All three underperform plain per-cell uniform crossover, and two fall *below random* at large budget. The cause is diversity: structural priors either collapse offspring variety into premature convergence (axis, path) or maintain it without exploiting it (aligned), while uniform crossover balances the two; a population-size sweep independently confirms that crossover's benefit is created by the diversity available to it. The lesson restates a known one — random search is a strong NAS baseline, structural cleverness rarely beats it — with a controlled mechanism on a real space. The isomorphism-correct oracle and all operators are one dependency-free C program; every number regenerates from one command.

1 Introduction

Neural architecture search (NAS) automates a network's shape; methods differ mainly in search strategy — RL [?], gradient relaxation [?], Bayesian optimization, random search, evolution [?]. Evolutionary NAS mutates and recombines architectures, training each candidate by back-propagation [?]. Two facts motivate us: plain random search is a strong baseline [?, ?], and the leading evolutionary method drops crossover entirely, keeping only mutation with aging [?].

We study two things on a real benchmark: whether crossover beats random search at all, and whether a crossover that *respects* the space's structure — NAS-Bench-101's cell factors into a wiring dimension and a labeling dimension — beats one that ignores it. The intuition for the second is strong: a graph's building blocks are its subgraphs and wiring, not arbitrary bits. It beats random, but the structure-aware operators are worse than the structure-blind one, reversing below random at scale. (The recombination logic comes from a self-modifying back-propagation engine [?]; we move fitness onto the table so training noise cannot confound the comparison.)

2 The space and the fitness oracle

A NAS-Bench-101 cell is a directed acyclic graph on up to seven nodes: node 0 input, node $n-1$ output, interior nodes labeled `conv1x1`/`conv3x3`/`maxpool3x3`. It has two orthogonal dimensions

*Independent researcher. Physics and computer science by education; a professional software engineer with a background in numerical optimization and machine learning. Reference implementation: `https://github.com/Anode1/SMBPANN`.

— *wiring* (the adjacency) and *labeling* (the ops) — and a derived path view. The benchmark tabulates the trained CIFAR-10 accuracy of all 423,624 cells, so fitness is a lookup.

The benchmark de-duplicates *isomorphic* graphs, so a recombined child is usually a relabeling of a catalogued cell rather than a verbatim match, and lookup needs a canonical form. We compute it by brute force: prune nodes lying on no input–output path, then take the lexicographically minimum adjacency-plus-labeling encoding over all $\leq 5! = 120$ interior-node permutations — exact and dependency-free. A self-test confirms it: over 5,000 cells both the verbatim graph and a random relabeling map to the same accuracy (5000/5000 each), and the 423,624 rows collapse to exactly 423,624 canonical keys (no collisions). This same canonicalization supplies the alignment operator's node correspondence. The table is extracted from the official `nasbench_only108.tfrecord` by a small C decoder (TFRecord framing, JSON, hand-parsed `ModelMetrics` protobuf); best test 94.32% / val 95.06% reproduce the published optima.

3 Operators

Every individual is a seven-node upper-triangular cell (input 0, output 6). All arms share one population loop, one light mutation (flip an edge or re-roll an interior op), elitism, and query budget; *only the crossover differs*, so any difference is attributable to it. Selection is on validation accuracy, reporting test. Four crossovers against a random control:

- **flat** — per-cell uniform: each adjacency bit and each op from a uniformly chosen parent. Structure-blind, maximum-diversity (and the "competing-conventions" strawman, since node k need not play the same role in both parents).
- **axis** — dimension-factored: the whole wiring from one parent, the whole labeling from the other.
- **aligned** — role-aligned: match the second parent's interior nodes to the first by role (op and in/out degree), then recombine *node modules* (each node's incoming edges and its op together from one aligned parent) — removing the competing-conventions problem before recombining.
- **path** — subgraph transplant: recombine whole input–output paths, each carrying its own parent's ops, added greedily under the nine-edge cap (the graph dimension).

Matched compute, paired. Each arm gets a fixed query budget; the random control spends the same on independent random cells; a child that prunes invalid or over-budget is a spent query scored zero, penalizing wasted queries honestly. In each trial the control and all arms share the *same* seed — an identical initial population — so a trial is a paired observation; we report paired win/loss and the Wilcoxon signed-rank z of (arm − baseline).

4 Results

Crossover beats random. At small population (20, elite 5, 800 seeds) every arm beats random at every budget (Wilcoxon $z = 2$–14). Unlike the six-gene NAS-Bench-201 cell, where crossover's edge is a fraction of a percent and vanishes at generous budget, the larger 101 space gives recombination something real to exploit.

Structure does not help; at scale it hurts. Table **??** scales to population 50, budget 20,000, 4000 paired seeds. **Flat** wins at *every* budget and beats each structured operator head-to-head; its edge over random shrinks as random saturates (z : $48 \to 46 \to 29 \to 13$) but never vanishes (2268/1322 paired wins even at 20,000). The structured operators worsen with compute: **axis** crosses from z=+30 to −28 (losing to random 1230/2559); **path** plateaus almost

Table 1: NAS-Bench-101, population 50, elite 12, 4000 paired seeds. Mean best test-accuracy (%), and Wilcoxon signed-rank z against random search (positive = beats random). Flat wins at every budget; *axis and path drop below random at scale*; aligned's edge decays to zero. Every arm shares each seed with the control.

budget	random	flat (z)	axis (z)	aligned (z)	path (z)
500	93.645	**93.998** (+48.0)	93.806 (+30.3)	93.917 (+41.8)	93.904 (+44.4)
2000	93.746	**94.072** (+45.8)	93.872 (+22.5)	94.009 (+39.8)	93.911 (+29.9)
8000	93.941	**94.085** (+29.2)	93.883 (−**10.2**)	94.036 (+19.7)	93.912 (−**6.1**)
20000	94.042	**94.088** (+12.5)	93.886 (−**27.6**)	94.044 (+1.9)	93.913 (−**26.0**)

Table 2: Population sweep at fixed budget 2000, 600 paired seeds (elite = pop/4; random control identical throughout, mean 93.743). Mean best test-accuracy (%) and Wilcoxon z vs random. Crossover's benefit is created by diversity: at pop 8 every arm loses to random; the gain rises monotonically with population.

pop	flat (z)	axis (z)	aligned (z)	path (z)
8	93.673 (−4.5)	93.652 (−5.7)	93.673 (−4.2)	93.526 (−12.6)
16	93.791 (+3.5)	93.715 (−1.8)	93.756 (+1.1)	93.706 (−2.7)
32	93.981 (+14.6)	93.823 (+5.9)	93.898 (+10.5)	93.840 (+7.3)
64	**94.122** (+19.3)	93.917 (+11.6)	94.064 (+17.2)	93.957 (+14.2)

immediately (93.904→93.913 over a 40× budget rise) and ends at z=−26; **aligned** decays from z=42 to 1.9, to break-even. So the structure-blind operator wins throughout, the two most structured (axis, path) fall below random at scale, and the intermediate one ties it. This is not a validity artifact: axis/aligned/path in fact yield *more* valid offspring (95–97%) than flat (90–96%).

Mechanism: productive diversity. Two independent measurements locate the cause. (i) Final-population genotypic spread (mean pairwise distance over 21 edges + 5 ops) shows two failure modes: the operators that end below random, axis and path, *collapse* the spread (1.8 and 1.3 vs flat's 2.1 at 20,000) — premature convergence, matching their plateaus; but aligned keeps the *most* spread (3.1, above flat) and still only ties random, its variety never concentrating on good cells. Flat holds an intermediate, productive spread and wins. (We had expected the cleaner "diversity ranks the operators"; the measurement falsifies it, and we report the falsification.) (ii) Varying *population* at fixed budget sweeps the diversity available to crossover, and the comparison moves with it (Table **??**): at population 8 every arm — flat included — loses to random ($z < 0$); the gain rises monotonically until at 64 all four beat it. Crossover here is better than random exactly when it has enough diversity *and* an operator that turns diversity into good offspring rather than collapsing or dissipating it.

5 Relation to prior work

Nothing here is new in framing. Random search as a strong NAS baseline is [**?**, **?**]; mutation-only evolutionary NAS is [**?**]; the competing-conventions problem the aligned operator targets motivates NEAT's historical markings [**?**]; NAS-Bench-101 [**?**] makes a study at this seed count possible. The contribution is a clean, reproducible *negative* result: on a real multi-dimensional NAS space, four structure-respecting crossovers all lose to plain uniform crossover, two to random search, with a diversity mechanism that explains why.

6 Discussion

The appealing idea that crossover should recombine along a search space's natural dimensions does not survive contact with the data: on NAS-Bench-101 it is beaten by structure-blind uniform crossover, the more structured the operator the sooner random search overtakes it, and the cause is diversity, not validity. **Limitations:** one tabulated cell space with lookup fitness (no training confound, but no transfer guarantee to larger or hierarchical spaces); `aligned` is one alignment and a better one may exist — but the burden is now on exhibiting one that beats uniform crossover, which four natural attempts do not. The natural next step is a larger, hierarchical space where recombination has more room to matter.

Reproducibility

Every number regenerates from [**?**] with one command. The fitness table is extracted by `nb101_extract.c` (`make nb101_extract`); the search, oracle, and all operators are `validation/nb101.c` (`make nb101`), configurable by the environment (`POP`, `ELITE`, `SEEDS`, `BUDGETS`) and self-testing its oracle with `-selftest`. Both build clean under `-std=c99 -pedantic` and run clean under AddressSanitizer and UBSan; a fixed xorshift generator makes every run reproducible.

References

[1] V. Gavrilov. *Backpropagation Feed-Forward Neural Networks.* Seminar in Artificial Intelligence, Open University of Israel, 1997 (revised digital edition, Zenodo 2026). `https://doi.org/10.5281/zenodo.20450526`

[2] J. Bergstra, Y. Bengio. Random Search for Hyper-Parameter Optimization. *JMLR* 13, 2012.

[3] B. Zoph, Q. V. Le. Neural Architecture Search with Reinforcement Learning. *ICLR* 2017.

[4] H. Liu, K. Simonyan, Y. Yang. DARTS: Differentiable Architecture Search. *ICLR* 2019.

[5] E. Real, A. Aggarwal, Y. Huang, Q. V. Le. Regularized Evolution for Image Classifier Architecture Search. *AAAI* 2019.

[6] T. Elsken, J. H. Metzen, F. Hutter. Neural Architecture Search: A Survey. *JMLR* 20, 2019.

[7] L. Li, A. Talwalkar. Random Search and Reproducibility for Neural Architecture Search. *UAI* 2019.

[8] K. O. Stanley, R. Miikkulainen. Evolving Neural Networks through Augmenting Topologies. *Evolutionary Computation* 10(2), 2002.

[9] C. Ying, A. Klein, E. Real, E. Christiansen, K. Murphy, F. Hutter. NAS-Bench-101: Towards Reproducible Neural Architecture Search. *ICML* 2019.

[10] SMBPANN reference implementation. `https://github.com/Anode1/SMBPANN`.